AI-POWERED SMART ORCHID CARE SYSTEM FOR VANDA CULTIVATION

INDIVIDUAL THESIS

Samadinee L. D. N.

(IT22083500)

Individual component: Growth Stage Classification and Bloom Prediction for Vanda Species

Group ID: R26-SE-018

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specialising in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Samadinee L. D. N.

Student ID: IT22083500

Signature:  _ Date: 2026/10/11

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Hansi De Silva		
Co-supervisor: Ms. Thilini Jayalath		

ABSTRACT

Vanda orchids are a valuable flower crop in Sri Lanka, yet growers still make two important judgements by eye: what growth stage each plant has reached, and when it will bloom. Getting either one wrong wastes fertiliser and can cost the grower a sale or a place on an export shipment. The systems that exist today were built for other orchids, group plants by age rather than by flowering stage, do not predict bloom, or depend on expensive instruments. In this research I built a growth stage classification and bloom prediction component for Vanda as part of an AI-powered smart orchid care system. A classifier built on a pretrained EfficientNetB0 network recognises six stages (vegetative, budding, pre-bloom, full bloom, wilting and seed formation) from an ordinary phone photograph and returns care advice suited to that stage. An open-vocabulary detector, OWLv2, allows the component to give a separate stage for each plant, flower bunch, bud and seed pod in a crowded photograph. A two-branch model then combines the photograph with temperature, humidity and light intensity to estimate the number of days until bloom. On 1,768 test images the classifier reached 0.940 accuracy and a macro F1 of 0.94, compared with a baseline of 0.196. On 1,839 test images the bloom model reached a mean absolute error of 0.92 days and a coefficient of determination of 0.938, compared with 4.86 days for predicting the average. Single-plant predictions came back in under one second, and 15 of the 16 API tests passed. The component is delivered through a FastAPI backend and a React Native mobile app, and the watering and pollination components of the system rely on its stage and bloom outputs. The results show that a phone photograph is enough to build a reliable, low-cost phenology tool for Vanda growers.

Keywords: Vanda orchid, growth stage classification, bloom prediction, transfer learning, EfficientNet

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Ms. Hansi De Silva and my co-supervisor, Ms. Thilini Jayalath, for their guidance, advice and patience throughout this research.

I am grateful to the Vanda growers who allowed me to photograph their plants on many visits and who explained how they read the growth and flowering of their orchids. Their knowledge shaped the six stages and the care advice used in this work.

I also thank my group members, Wickramasinghe D. P., Rathnamalala D. M. J. P. and Vithanage P. V. P. M., for their teamwork in building the shared backend, database and mobile application, and the academic staff of the Sri Lanka Institute of Information Technology for their support.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i

ABSTRACT ii

ACKNOWLEDGEMENTS iii

TABLE OF CONTENTS iv

LIST OF FIGURES ix

LIST OF TABLES xi

LIST OF ABBREVIATIONS xiii

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 1

1.3 Problem Statement 2

1.4 Research Gap 3

1.5 Aim and Objectives 4

1.6 Research Questions 4

1.7 Scope, Limitations and Assumptions 5

1.8 Individual Contribution 5

1.9 Thesis Structure 6

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review Method and Scope 7

2.2 Domain and Theoretical Foundations 7

2.2.1 The Vanda flowering cycle 7

2.2.2 Convolutional neural networks 8

2.2.3 Transfer learning 8

2.2.4 EfficientNet 8

2.2.5 Regression with images and sensor data 8
2.2.6 Object detection 9
2.3 Related Research 9
2.4 Existing Systems, Products and Techniques 10
2.5 Gap Synthesis and Conceptual Framework 10
3 RESEARCH METHODOLOGY 12
3.1 Research Design 12
3.2 Data, Participants and Study Materials 12
3.2.1 Growth stage dataset 12
3.2.2 Bloom record 13
3.2.3 Participants 14
3.3 Experimental Procedure 14
3.3.1 Data cleaning 14
3.3.2 Splitting the data 15
3.3.3 Image preparation 15
3.3.4 Training procedure 15
3.4 Measures and Evaluation Metrics 16
3.5 Data Analysis 17
3.6 Ethics, Privacy, Security and Responsible AI 17
3.7 Reproducibility and Research Artefacts 18
4 PRODUCT AND SOLUTION DESIGN 19
4.1 Requirements 19
4.2 Component Architecture within the Complete Solution 20
4.3 Design Alternatives and Decisions 20
4.4 Detailed Design 21
4.4.1 Growth stage classifier 21

4.4.2 Bloom prediction model 22
4.4.3 Multi-object pipeline 22
4.4.4 API design 23
4.4.5 Request flow 23
4.4.6 User interface design 24
4.4.7 Care protocol 24
4.5 Standards and Professional Practice 25
5 IMPLEMENTATION, INTEGRATION AND TESTING 26
5.1 Development Environment and Technology Stack 26
5.2 Component Implementation 26
5.2.1 Data preparation code 27
5.2.2 Training code 27
5.2.3 Prediction code 28
5.2.4 Backend routes 28
5.2.5 Mobile screen 28
5.3 Integration with the Other Components 28
5.4 Verification and Software Testing 29
5.4.1 Testing strategy 29
5.4.2 Test environment 29
5.4.3 Functional and performance test cases 29
5.4.4 User interface test cases 30
5.5 Deployment and Operation 31
6 RESULTS AND VALIDATION 32
6.1 Research Results 32
6.1.1 Growth stage classification 32
6.1.2 Bloom prediction 33

6.2 Product and Technical Results 35
6.3 User and Stakeholder Validation 36
6.4 Integration Results 36
6.5 Summary of Findings 36
7 DISCUSSION 37
7.1 Interpretation of Findings 37
7.2 Comparison with Literature, Baselines and Existing Solutions 38
7.3 Research Contribution 38
7.4 Product Contribution 38
7.5 Limitations and Threats to Validity 39
7.6 Implications 39
8 COMMERCIALISATION AND SUSTAINABILITY 41
8.1 Value Proposition and Target Users 41
8.2 Market and Competitive Position 41
8.3 Business and Sustainability Model 41
8.4 Scalability, Intellectual Property and Compliance 42
8.5 Contribution to the Sustainable Development Goals 42
9 CONCLUSION AND FUTURE DIRECTIONS 43
9.1 Conclusion 43
9.2 Achievement of Objectives 43
9.3 Individual Research and Product Contributions 43
9.4 Future Directions 44
REFERENCES 45
GLOSSARY 47
APPENDICES 48
Appendix A Extended Results and Technical Details 48

A.1 Sample API answers 48
A.2 Growth stage classifier code 50
A.3 Bloom prediction model code 51
A.4 Source code repository 51
A.5 Application screenshots 52
Appendix B Academic Integrity and AI Use Disclosure 55

LIST OF FIGURES

Figure 1.1: The six Vanda growth stages recognized by the component and the feed used at each stage (original in color) 2
Figure 2.1: Conceptual framework of the research 11
Figure 3.1: Steps of the research process 12
Figure 3.2: Number of images in each growth stage 13
Figure 3.3: Spread of the number of days until bloom in the bloom record 14
Figure 4.1: Architecture of the growth stage classification and bloom prediction component 20
Figure 4.2: Sequence of a growth stage request 24
Figure 6.1: Recall of each stage on the test set; the dashed line is the majority-class baseline of 0.196 32
Figure 6.2: Confusion matrix of the growth stage classifier on the test set, as counts and as proportions of each row 33
Figure 6.3: Training and validation loss and accuracy of the growth stage classifier; phase two starts at epoch 50 33
Figure 6.4: Predicted against actual days until bloom on the test set; the dashed line is a perfect prediction 34
Figure 6.5: Training and validation loss and mean absolute error of the bloom prediction model 35
Figure A.1: Response of POST /api/v1/growth/identify for the budding test photograph (original in color) 48
Figure A.2: Response of POST /api/v1/bloom/predict for the same photograph at 30 °C, 74 per cent humidity and 4,207 lux (original in color) 49
Figure A.3: Growth Stage screen before a photograph is chosen, showing the Gallery and Camera options and the live sensor readings 52
Figure A.4: Selected photograph of a budding Vanda and the predicted stage, Budding Stage, at 99.9 per cent confidence 53
Figure A.5: Stage result with the confidence bar, the top three predictions and the bloom date inputs 54

Figure A.6: Bloom date prediction of six days, the care protocol for the budding stage and the Share Results button..... 55

LIST OF TABLES

Table 1.1: The four components of the integrated system 1
Table 1.2: Summary of my individual contribution 5
Table 2.1: Options available to a Vanda grower today 10
Table 2.2: Comparison of the closest systems with the proposed component 11
Table 3.1: Growth stage classes and number of images 12
Table 3.2: Fields of the bloom record 13
Table 3.3: Size of the training, validation and test sets 15
Table 3.4: Main training settings 16
Table 3.5: Main research artefacts 18
Table 4.1: Functional requirements 19
Table 4.2: Non-functional requirements 19
Table 4.3: Main design decisions 21
Table 4.4: Layers of the growth stage classifier 21
Table 4.5: Structure of the bloom prediction model 22
Table 4.6: Endpoints of the component 23
Table 4.7: Care advice returned for each stage 24
Table 5.1: Technology stack of the component 26
Table 5.2: Main source files of the component 27
Table 5.3: Testing levels 29
Table 5.4: API test cases and results 30
Table 5.5: User interface test cases 31
Table 6.1: Growth stage classifier: results for each stage on the test set 32
Table 6.2: Bloom prediction results on the test set 34
Table 6.3: Summary of the main results 36
Table 7.1: Comparison with existing work 38
Table 8.1: Target users and value 41
Table 8.2: Competitive position 41
Table 8.3: Possible business model 42
Table 8.4: Contribution to the Sustainable Development Goals [16] 42
Table 9.1: Achievement of the specific objectives 43

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
F1	Harmonic mean of precision and recall
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
MAE	Mean Absolute Error
NFR	Non-Functional Requirement
NPK	Nitrogen, Phosphorus and Potassium
OWLv2	Open-World Localization detector, version 2
ReLU	Rectified Linear Unit
REST	Representational State Transfer
RMSE	Root Mean Squared Error
RQ	Research Question
R^2	Coefficient of determination
SDG	Sustainable Development Goal
SO	Specific Objective
YOLO	You Only Look Once (object detector family)

1 INTRODUCTION

1.1 Overall Project Context

Vanda orchids are among the most valuable flowers cultivated in Sri Lanka. Healthy, flowering plants command high prices, and many growers sell the plants themselves while also supplying cut flowers to local buyers and export companies. Despite this, many day-to-day decisions in a Vanda shade house are still made based on visual observation. A grower might examine a leaf to assess for disease, study the plant to gauge its growth stage, and check the sky to decide whether watering is necessary. None of these decisions are documented; they rely entirely on experience, making it difficult to pass this knowledge on to a newcomer.

Our team project an AI-powered smart orchid care system for Vanda cultivation aims to support these decision-making processes using machine learning. The system consists of four components, each addressing a question frequently asked by Vanda growers. All four components share a single FastAPI backend, an integrated Firebase database, and a single mobile application, ensuring that the grower needs only one app. Table 1.1 lists the components and specifies who is responsible for each.

Table 1.1: The four components of the integrated system

No.	Component	Question it answers	Owner
C1	Hybrid pollination and compatibility analysis	Is this plant fit to breed from, and with which partner?	Wickramasinghe D. P. (IT22065230)
C2	Disease detection, severity and treatment	Is this plant diseased, and how badly?	Rathnamalala D. M. J. P. (IT22064936)
C3	Smart watering and automated fertilisation	When and how much should this plant be watered and fed?	Vithanage P. V. P. M. (IT22062642)
C4	Growth stage classification and bloom prediction	Which stage has this plant reached, and when will it bloom?	Samadinee L. D. N. (IT22083500)

1.2 Individual Component Context

Component 4 answers two questions: which growth stage the Vanda has reached and how many days remain before it blooms. Both are important, as the answers determine the grower's next steps.

The growth stage dictates the care a plant requires. A plant producing only leaves needs a balanced fertilizer, whereas a plant with new shoots benefits from a bloom

booster. When in full bloom, it requires light support, while a plant with fading flowers needs less water and a return to a balanced fertilizer. Misinterpreting the stage can waste an entire nutrient cycle and delay the next flowering. Figure 1.1 illustrates the six stages identified by the component and the corresponding fertilizer for each.

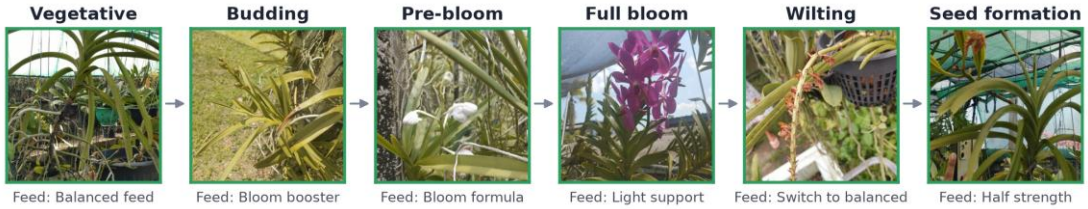


Figure 1.1: The six Vanda growth stages recognized by the component and the feed used at each stage (original in color)

On the other hand, the flowering date determines when the plant can be sold. A Vanda orchid is most valuable during the brief window when its flowers are fresh and fully open. Knowing the flowering date in advance allows a grower to plan sales, prepare plants for export, and organize staff. Without this knowledge, the flowers might open before a buyer is ready or after a shipment has already been dispatched.

The component provides both answers based on a standard smartphone photo. To determine the flowering date, the grower also inputs three simple readings: temperature, humidity, and light intensity. The result is then displayed on the mobile app, accompanied by simple care instructions.

1.3 Problem Statement

Currently, Vanda growers assess growth stages and flowering times visually—an approach that has four drawbacks. It lacks consistency: two growers might disagree when looking at the same plant, and a single grower might make different assessments on a busy day compared to a quiet one. It leaves no record, meaning there is no historical data to refer back to. It takes years to master, placing new growers at a disadvantage. It is a slow process when a shade house contains hundreds of plants.

Existing technology does not meet this need for Vandas. Current systems are designed for other types of orchids, rely on plant age rather than flowering stage, fail

to predict flowering dates, or require expensive scientific equipment. Chapter 2 reviews these systems in detail.

Therefore, the problem addressed by this research is the lack of a simple, low-cost tool that can inform Vanda growers about a plant's growth stage and the time remaining until it blooms, while also indicating the specific care the plant requires at that stage.

1.4 Research Gap

The literature review in Chapter 2 identified two systems that come close to this work. One is a 2025 study that monitors orchid growth and predicts bloom using hyperspectral cameras, fluorescence sensors and machine vision [11]. The other is OrchiZen, a smart farming system for Dendrobium, Vanda and Phalaenopsis growers [12]. Both are valuable, but they leave four gaps:

- **No Vanda focus.** The optical study was designed for Phalaenopsis and does not take account of how Vanda grows: a single upward stem, aerial roots, and flowering that can happen several times a year.
- **Age instead of flowering stage.** OrchiZen groups plants by age (baby, child, young, mature and adult), but a plant's age says nothing about whether it is about to flower.
- **No bloom prediction in OrchiZen, and no post-bloom stages in the optical study.** The optical study does not model repeated flowering, and it stops following the plant once it has flowered, which is exactly when care needs to change and seeds need to be collected.
- **Expensive equipment.** The optical study needs hyperspectral and fluorescence instruments that a small grower does not own.

This research addresses these gaps with a tool built specifically for Vanda. It recognises the stages of the flowering cycle from a normal phone photograph, predicts the bloom date, and delivers care advice through a mobile app.

1.5 Aim and Objectives

Aim. To build and test an artificial intelligence (AI) component that identifies the growth stage of a Vanda orchid from a photograph, predicts how many days remain until it blooms, and gives the grower stage-specific care advice through the group's mobile application.

Specific objectives. The aim is broken into six specific objectives (SO):

- **SO1:** Build a labelled image dataset of six Vanda growth stages: vegetative, budding, pre-bloom, full bloom, wilting and seed formation.
- **SO2:** Train a deep learning model that classifies the six stages, and test it on images it has never seen against a simple baseline.
- **SO3:** Build a bloom record that links each photograph to temperature, humidity, light intensity and the number of days until bloom, and train a model that predicts the number of days.
- **SO4:** Find and classify several plants, flower bunches, buds and seed pods in one photograph.
- **SO5:** Make both models available through a Representational State Transfer (REST) application programming interface (API) and use them in the mobile application with stage-specific care advice.
- **SO6:** Share the growth stage and bloom date with the other components of the system.

1.6 Research Questions

The research tries to answer three research questions (RQs):

- **RQ1:** Can a transfer learning model tell the six Vanda growth stages apart from a phone photograph, and how much better is it than always guessing the most common stage?
- **RQ2:** Can a photograph together with temperature, humidity and light intensity predict the number of days until bloom with an error small enough to plan sales?
- **RQ3:** Can the results reach the grower quickly, within a few seconds, and in a form that tells the grower what to do?

1.7 Scope, Limitations and Assumptions

Scope. The component is intended for Vanda orchids grown in Sri Lankan shade houses. It recognizes six stages of the flowering cycle and predicts the days until bloom up to roughly three weeks in advance, using ordinary phone photographs and three readings entered by the grower.

Limitations. The germination and seedling stages are left out because there were not enough usable photographs of them. The component does not yet estimate how long flowers will last or when seeds should be collected. The stage model will assign a stage to any photograph, even one that does not show an orchid, and the multi-object detector runs slowly on an ordinary computer. Section 7.5 discusses these limits further.

Assumptions. It is assumed that the grower takes the photograph in daylight, from a normal distance, with the plant clearly in view, and that the temperature, humidity and light readings are taken at the same time and entered correctly.

1.8 Individual Contribution

I designed, built and tested every part of Component 4, as summarized in Table 1.2. Shared parts of the system, such as the backend server and the database, were built jointly with the group.

Table 1.2: Summary of my individual contribution

Area	My work
Data	Collected and labelled the growth stage photographs and the bloom record; wrote a cleaning script that fixes unreadable files and removes duplicate photographs
Models	Designed and trained the six-class growth stage classifier and the two-branch bloom prediction model
Object detection	Built the pipeline that finds each plant, flower bunch, bud and seed pod in a photograph and classifies each one
Backend	Wrote the growth stage and bloom prediction API routes in the shared FastAPI backend
Mobile app	Built the Growth Stage screen with photograph capture, results, care advice, bloom prediction and sharing
Integration	Provided the stage and bloom outputs used by the watering and pollination components
Testing	Evaluated both models on held-out test data and ran 16 API test cases

1.9 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 reviews literature and existing systems and sets out the research gap. Chapter 3 describes the research method, the data and the evaluation measures, and Chapter 4 presents the requirements and design of the component. Chapter 5 explains how the component was built, connected to the other components and tested, and Chapter 6 reports the results. Chapter 7 discusses what those results mean, compares them with other work and acknowledges the limitations. Chapter 8 covers commercialization and sustainability, and Chapter 9 concludes with suggestions for future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

I searched IEEE Xplore, Google Scholar and the SLIIT research repository using keywords such as “orchid growth stage”, “orchid bloom prediction”, “plant phenology deep learning”, “transfer learning plant images”, “EfficientNet agriculture” and “open-vocabulary object detection”. Studies were included if they dealt with image-based plant analysis, orchid cultivation, or the deep learning methods used in this work. Work on orchid disease was left to Component 2, and work on irrigation to Component 3.

The review is organized in four parts. Section 2.2 covers the background: the Vanda flowering cycle and the deep learning methods involved. Section 2.3 reviews related research, and Section 2.4 looks at the systems and techniques growers can use today. Section 2.5 draws these findings together to set out the research gap and the conceptual framework of this work.

2.2 Domain and Theoretical Foundations

2.2.1 The Vanda flowering cycle

Vanda is a monopodial orchid, meaning it grows from a single stem that keeps extending upwards rather than from a row of bulbs as many other orchids do. Its thick roots hang freely in the air, taking in water and nutrients from the surrounding air and from regular watering. A healthy Vanda can flower several times a year rather than only once in a fixed season.

For this reason, a calendar is a poor guide for a Vanda grower, who instead has to watch each plant individually as it moves through a cycle of stages. During the vegetative stage the plant produces new leaves and roots. In the budding stage, small buds appear on a flower spike, and by the pre-bloom stage these buds have swollen and begun to open. The flowers are fully open in the full bloom stage and then fade and drop in the wilting stage. If the flowers were pollinated, a seed pod forms and develops during the seed formation stage. The plant then returns to vegetative growth and the cycle begins again.

2.2.2 Convolutional neural networks

A convolutional neural network (CNN) is a deep learning model designed for images. It slides small filters across an image, picking up simple patterns such as edges and colors in its early layers and more complex ones, such as leaves, buds and flowers, in later layers. A final classification layer converts these patterns into a probability for each class. CNNs have become the standard approach for recognizing objects in photographs.

2.2.3 Transfer learning

Training a CNN from scratch requires millions of labelled images, and transfer learning is a way around this. The network is first trained on a very large general-purpose image collection such as ImageNet, which contains more than a million photographs of everyday objects [2]. Its learned layers are then reused for a new task, and only a small new section of the network is trained on the new images [3]. Yosinski et al. showed that the early layers of such networks learn general features useful for many tasks, whereas the later layers learn features specific to the original task [4]. This finding underpins a widely used two-step method, which this thesis follows: the new layers are trained first while the original layers stay frozen, and then the last few original layers are unfrozen and trained gently with a small learning rate.

2.2.4 EfficientNet

EfficientNet is a family of CNNs that scale the depth, width and input size of the network together in a balanced way [1]. Its smallest member, EfficientNetB0, works with 224 by 224 pixel images and has roughly five million parameters, yet it matches the accuracy of much larger networks while staying small and fast. That makes it well suited to a model that runs on a modest server and has to respond to a phone within a few seconds.

2.2.5 Regression with images and sensor data

Predicting the number of days until bloom is a regression task, because the answer is a number rather than a class. A model can draw on two kinds of input to do this. An image branch, such as a CNN, captures what the plant looks like, while a second branch of simple dense layers handles numeric readings such as temperature and humidity. The outputs of the two branches are combined and fed into a final layer

that produces the number. Such a design is usually called a two-branch or multi-input model.

2.2.6 Object detection

An object detector locates objects in an image and draws a box around each one. Conventional detectors such as the You Only Look Once (YOLO) family need many training images in which every object has been boxed by hand. Open-vocabulary detectors avoid this requirement. OWLv2 (Open-World Localization, version 2), for example, can find objects described in plain text, such as “orchid flower bud”, because it was trained on an enormous collection of images and their captions [7]. As a result, it can be used before a single orchid box has been drawn.

2.3 Related Research

Deep learning in agriculture. Kamilaris and Prenafeta-Boldú reviewed the use of deep learning in agriculture and found that CNNs outperform older methods in crop classification, weed detection, fruit counting and yield estimation [5]. Mohanty et al. applied transfer learning to recognize plant diseases from leaf photographs and achieved high accuracy, showing that pretrained CNNs can learn plant features from a moderate number of images [6]. Together, these studies support the choice of a pretrained CNN for this work.

Multi-modal optical sensing for orchids. Sungheetha et al. combined hyperspectral imaging, fluorescence spectroscopy and machine vision to monitor orchid growth on commercial farms in Asia [11]. Their system measures quantities such as photosynthetic efficiency, leaf area and early signs of stress, and reports bloom predictions 21 days in advance. However, it is aimed at Phalaenopsis and relies on expensive optical instruments. It does not account for the way Vanda grows, does not model repeated flowering, and stops following the plant once it has flowered.

OrchiZen. Wijendra et al. developed OrchiZen, a smart farming system for Dendrobium, Vanda and Phalaenopsis growers [12], with modules for irrigation, disease treatment, species selection, lighting and fertilizer. The fertilizer module identifies a plant’s growth stage using a custom YOLO detector combined with decision tree and nearest neighbor classifiers, and then recommends quantities of

nitrogen, phosphorus and potassium (NPK) fertilizer, calcium nitrate and potassium sulphate. Its stages (baby, child, young, mature and adult), however, describe the age of the plant, and the system does not predict the bloom date.

Open-vocabulary detection. Minderer et al. showed that a detector trained on very large collections of web images can find objects from text prompts without any task-specific boxes [7]. This means orchid parts can be located in crowded photographs right from the start of a project.

2.4 Existing Systems, Products and Techniques

A Vanda grower looking for help today has three main options, which are compared in Table 2.1.

Table 2.1: Options available to a Vanda grower today

Option	What it does	Main weakness for this problem
Manual inspection by an experienced grower	Reads stage and expected bloom from the plant	Not consistent, not recorded, takes years to learn
General plant identification apps	Name the plant species from a photo and give general care text	Do not identify the growth stage or predict bloom
Research systems with scientific sensors	Measure plant physiology and stress precisely	Need expensive instruments; not built for Vanda; not in a grower app
Farm management systems such as OrchiZen	Combine sensors and models for several tasks	Use plant age, not flowering stage; no bloom prediction

None of these options can give a Vanda grower a growth stage, a bloom date and care advice from a phone photograph.

2.5 Gap Synthesis and Conceptual Framework

Table 2.2 draws the review together by comparing the two closest systems with the component developed in this research.

Table 2.2: Comparison of the closest systems with the proposed component

Feature	Optical sensing [11]	OrchiZen [12]	This component
Built for Vanda	No (Phalaenopsis)	Partly (three genera)	Yes
Stages follow the flowering cycle	No	No (age groups)	Yes (six stages)
Bloom date prediction	Yes	No	Yes
Stages after flowering	No	No	Yes (wilting, seed formation)
Equipment needed	Hyperspectral and fluorescence sensors	Camera and sensors	Phone camera and three readings
Several plants in one photo	Not reported	Detector used for features	Yes, stage for each object
Care advice in a mobile app	No	Fertilizer amounts	Yes

Figure 2.1 shows the conceptual framework of the research. The grower supplies a photograph and, for bloom prediction, three readings. The models turn these inputs into a stage, a confidence value and a bloom date, and the component adds care advice. The grower then uses this output to decide which feed to apply, when to sell and when to pollinate.

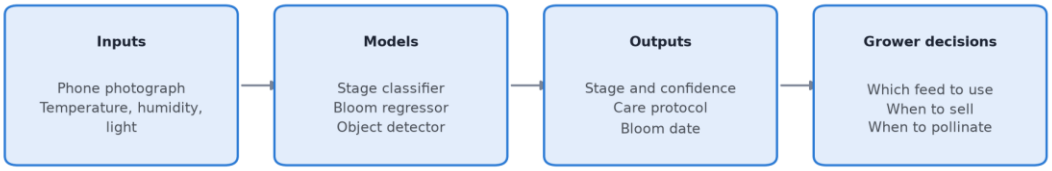


Figure 2.1: Conceptual framework of the research

3 RESEARCH METHODOLOGY

3.1 Research Design

This research follows a quantitative, experimental design. I built a software artefact, the growth stage and bloom prediction component, and measured its performance on data it had not seen during training. To make the real value of each model clear, the results were compared against simple baselines. Figure 3.1 outlines the steps of the research.



Figure 3.1: Steps of the research process

The work began with an investigation of the problem, through conversations with growers and a review of the literature. Photographs and bloom records were then collected and prepared, after which two models were trained and evaluated. Finally, the models were connected to the backend, the mobile app and the other components, and the complete component was tested.

3.2 Data, Participants and Study Materials

3.2.1 Growth stage dataset

The growth stage dataset consists of 11,562 images taken on 35 separate days in Vanda shade houses, each labelled with one of six stages. Table 3.1 describes the stages and gives the number of images for each, and Figure 3.2 presents the same figures as a chart.

Table 3.1: Growth stage classes and number of images

Stage	What the grower sees	Images	Test images
Vegetative	Leaves and aerial roots growing; no buds	1,795	280
Budding	Flower buds first appear on the spike	1,944	300
Pre-Bloom	Buds swell and begin to open	2,102	311
Full Bloom	Flowers fully open	2,286	346
Wilting	Flowers fade, dry and drop	1,411	220
Seed Formation	Seed pod develops after flowering	2,024	311

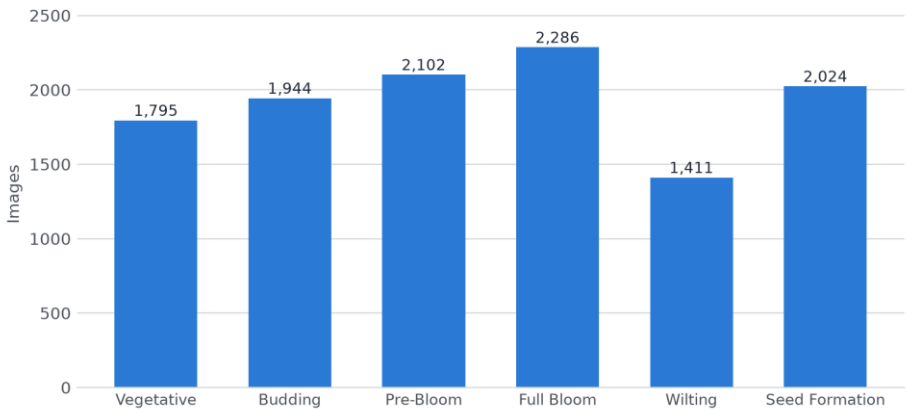


Figure 3.2: Number of images in each growth stage

The classes are reasonably balanced: the largest, full bloom, is about 1.6 times the size of the smallest, wilting. This matters because it prevents a model from scoring well simply by always choosing the same stage. A seventh stage, germination, was originally planned but was dropped from training because there were too few usable photographs.

3.2.2 Bloom record

The bloom record was collected on 12 days between 12 July and 4 August 2026. For each photograph, a spreadsheet row records the temperature, humidity and light intensity at the time it was taken, along with the number of days from that date until the plant bloomed. In total the record contains 12,260 images. The days until bloom range from 0 to 23, with a mean of 6.2 days and a median of 4 days. Table 3.2 lists the fields, and Figure 3.3 shows how the days until bloom are distributed.

Table 3.2: Fields of the bloom record

Field	Unit	Use in the model
Image	Photograph file	Input to the image branch
Temperature	Degrees Celsius	Input to the sensor branch
Humidity	Per cent relative humidity	Input to the sensor branch
Light intensity	Lux	Input to the sensor branch
No of days until bloom	Whole days (0 to 23)	Value the model predicts
Capture date	Date	Turns the days into a calendar date

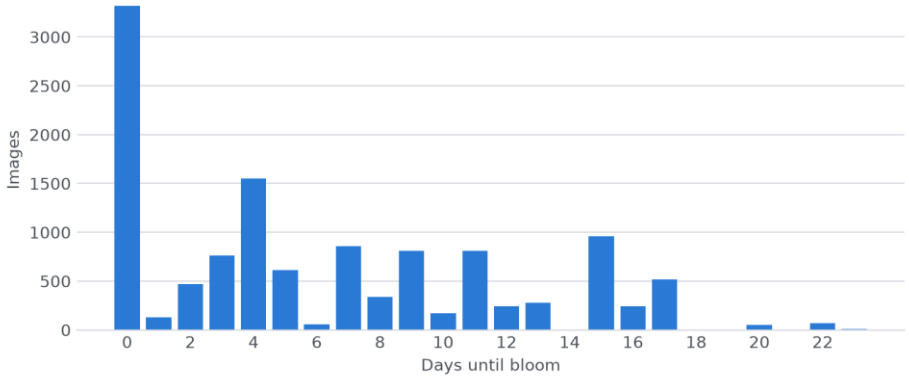


Figure 3.3: Spread of the number of days until bloom in the bloom record

A large number of images have a value of zero days, meaning the plant was already in bloom, while values above 17 days are uncommon. As Chapter 6 shows, this affects the results at the long end of the range.

3.2.3 Participants

Vanda growers gave their permission for their plants to be photographed and explained how they judge growth and flowering. No personal data about the growers was collected, and the photographs show only plants, pots and shade house structures.

3.3 Experimental Procedure

3.3.1 Data cleaning

Before training, I ran a cleaning script over the growth stage folders to fix two problems. First, some photographs had been saved as .jfif or .webp files, which the training program could not read, so they were silently skipped and never reached the model; the script saved a .jpg copy of each of these. Second, some identical photographs appeared in two different stage folders, which effectively gave the model two different answers for the same picture. The script detected these duplicates by comparing their digital fingerprints (hashes) and moved them into a separate quarantine folder for review rather than deleting them. Altogether, 780 files were quarantined.

3.3.2 Splitting the data

Each dataset was split into training, validation and test sets in a 70:15:15 ratio. The training set is used to teach the model, and the validation set is used during training to decide when to stop and when to lower the learning rate. The test set is held back and used only once, at the very end, to measure the final results. The growth stage data was split separately within each stage so that every set has the same mix of stages, while the bloom images were split at random. Table 3.3 gives the sizes of each set.

Table 3.3: Size of the training, validation and test sets

Model	Training	Validation	Test	Total
Growth stage classifier	8,049	1,745	1,768	11,562
Bloom prediction model	8,581	1,840	1,839	12,260

3.3.3 Image preparation

Every image is resized to 224 by 224 pixels, the input size of EfficientNetB0, and passed through the standard EfficientNet preprocessing step. During training, each image is also altered slightly at random every time it is used: it may be flipped, rotated by up to 20 degrees, or made a little brighter, darker or higher in contrast. This encourages the model to concentrate on the plant itself rather than on the exact framing or lighting of a photograph. These random changes are not applied to the validation and test images. For the bloom model, the three readings are scaled to a common range using values calculated from the training set alone.

3.3.4 Training procedure

Both models were trained in two phases on a cloud graphics processing unit (GPU) through Google Colab. In the first phase, the EfficientNetB0 layers were frozen and only the new layers were trained. In the second, the final layers of EfficientNetB0 were unfrozen and the whole model was trained further at a much lower learning rate. Training stopped automatically once the validation loss stopped improving, and the best weights were retained. Table 3.4 lists the main training settings.

Table 3.4: Main training settings

Setting	Growth stage classifier	Bloom prediction model
Pretrained base	EfficientNetB0 (ImageNet)	EfficientNetB0 (ImageNet)
Input	224 × 224 image	224 × 224 image + 3 readings
Loss function	Categorical cross-entropy	Mean squared error
Optimiser and learning rate	Adam [8], 0.001	Adam [8], 0.001
Batch size	32	16
Layers unfrozen in phase two	Last 30	Last 10
Learning rate in phase two	0.0001 (one tenth)	0.00005 (one twentieth)
Early stopping patience	10 epochs	5 epochs
Class weights	Yes, from class sizes	Not applicable
Epochs trained	68	49
Training time	41 minutes	35 minutes

3.4 Measures and Evaluation Metrics

Classification measures. Four common measures are used for the growth stage model. *Accuracy* is the proportion of test images whose stage is predicted correctly. *Precision* for a stage is the proportion of images predicted as that stage that actually belong to it. *Recall* for a stage is the proportion of images of that stage that the model correctly identifies. The *F1 score* balances precision and recall: F1 = 2 × precision × recall / (precision + recall). The *macro F1* is the simple average of the F1 scores for the six stages, so every stage carries equal weight. Finally, a *confusion matrix* shows which stages are confused with which.

Regression measures. Three measures are used for the bloom model. The *mean absolute error (MAE)* is the average size of the error in days. The *root mean squared error (RMSE)* is similar but penalizes large errors more heavily. The *coefficient of determination (R^2)* indicates how much of the variation in days until bloom the model explains, where 1 is perfect and 0 is no better than always predicting the average.

Baselines. A result is only meaningful when set against a simple alternative. For the growth stage model, the baseline always predicts the most common stage in the test set. For the bloom model, it always predicts the average number of days from the training set.

System measures. For the API, I measured the response time from sending a request to receiving the answer, and checked whether each endpoint returns the correct result and the correct error codes.

3.5 Data Analysis

Once training was complete, each model was run on its test set. For the growth stage model, scikit-learn [14] was used to produce a classification report (precision, recall and F1 for each stage) and a confusion matrix. For the bloom model, I calculated the MAE, RMSE and R^2 and plotted predicted against actual days. The training and validation curves of both models were also plotted to check for overfitting, and each result was compared with its baseline. Finally, the API was tested with real test photographs and the response times were recorded.

3.6 Ethics, Privacy, Security and Responsible AI

Ethics and consent. Photographs were taken only with the shade house owners' permission, and the study involved no experiments on people.

Privacy. The component works only with photographs of plants and three environmental readings. It does not collect growers' names, phone numbers or locations, and any person who appeared in a photograph by accident was excluded from analysis.

Security. The backend accepts only image files of known types and rejects any other upload. Uploaded images are kept only as long as it takes to make a prediction and are then deleted, and secret keys for shared services are kept out of the code repository.

Responsible AI. The component offers advice rather than instructions. Each prediction is accompanied by a confidence value and the next two most likely stages, so the grower can see when the model is uncertain. The thesis also reports the weaknesses of the models openly, including which stages get confused and where the bloom prediction is less accurate.

3.7 Reproducibility and Research Artefacts

All code, settings and trained models are stored in the group's Git repository. The exact list of files used for training, validation and testing is saved alongside each model so that the same split can be checked again later, and a fixed random seed (42) is used for splitting. Table 3.5 lists the main artefacts.

Table 3.5: Main research artefacts

Artefact	Location in the repository
Growth stage settings and stage information	ml-models/growth_stage/src/config.py
Data cleaning script	ml-models/growth_stage/src/clean_dataset.py
Training scripts	ml-models/growth_stage/src/train.py, ml-models/bloom_prediction/src/train.py
Trained models	ml-models/growth_stage/models/vanda_growth_model.h5, ml-models/bloom_prediction/models/vanda_bloom_model.h5
Split lists	models/train_split.csv, val_split.csv, test_split.csv in each model folder
Evaluation reports and plots	classification_report.txt, confusion_matrix.png, evaluation_report.txt, prediction_scatter.png
Colab training notebooks	notebooks/colab_train.ipynb in each model folder

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements were drawn from conversations with growers, the needs of the other components, and literature. Table 4.1 lists the functional requirements (what the component must do), and Table 4.2 lists the non-functional requirements, or NFRs (how well it must do it).

Table 4.1: Functional requirements

ID	Requirement
FR1	The grower can take a photograph with the phone camera or choose one from the gallery.
FR2	The component identifies one of six growth stages from the photograph.
FR3	The component shows the confidence of the prediction and the two next most likely stages.
FR4	The component shows care advice (water, light, temperature, fertiliser) for the predicted stage.
FR5	The grower can enter temperature, humidity and light intensity and get the number of days until bloom and the bloom date.
FR6	The component can find several plants, flower bunches, buds and seed pods in one photograph and give a stage for each.
FR7	The grower can share the result as text.
FR8	The component provides the stage to the watering component and the bloom date to the pollination component.

Table 4.2: Non-functional requirements

ID	Quality	Requirement
NFR1	Accuracy	The stage model must clearly beat the majority-class baseline on test data.
NFR2	Accuracy	The bloom model must clearly beat the average-value baseline on test data.
NFR3	Performance	A single-plant prediction must return in less than five seconds.
NFR4	Usability	The result must be understandable by a grower without technical knowledge.
NFR5	Reliability	Wrong inputs (non-image files, missing readings) must give a clear error, not a crash.
NFR6	Security	Only image files are accepted and uploaded images are deleted after use.
NFR7	Maintainability	Models and settings can be replaced without changing the API or the app.
NFR8	Cost	The component must work with a phone camera and no special hardware.

4.2 Component Architecture within the Complete Solution

Figure 4.1 shows the architecture of the component. The grower works from the Growth Stage screen of the mobile app, which sends the photograph, along with the three readings when bloom prediction is requested, to the FastAPI backend. The backend validates the file and passes it to the appropriate model: the growth stage classifier, the multi-object pipeline or the bloom prediction model. It then returns a JavaScript Object Notation (JSON) response for the app to display.

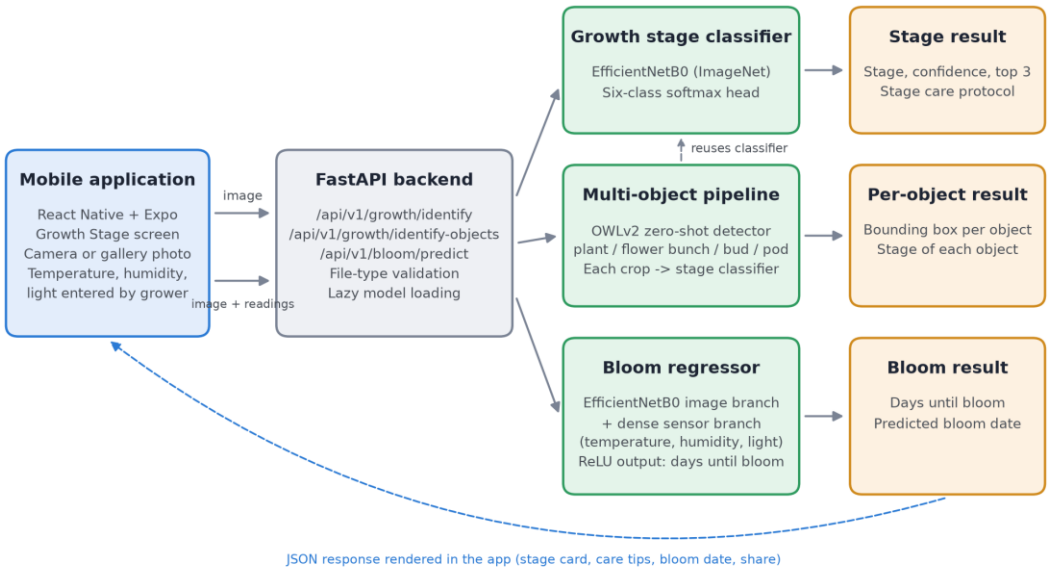


Figure 4.1: Architecture of the growth stage classification and bloom prediction component

This is one of four components in the complete system, and it shares the same backend and app with the other three. Two of those components also rely on its outputs: the watering and fertilization component (C3) uses the growth stage to choose the right fertilizer, and the pollination component (C1) uses the bloom date to know when a plant will have open flowers.

4.3 Design Alternatives and Decisions

A number of design choices were made over the course of the project. Table 4.3 sets out the main alternatives and the reasoning behind each decision.

Table 4.3: Main design decisions

Decision	Alternatives considered	Choice and reason
Base network	EfficientNetB3, MobileNetV2, ResNet50	EfficientNetB0: designed for 224 × 224 images, small and accurate. B3 is designed for larger images and overfitted more on this data.
Training method	Train from scratch; fine-tune all layers at once	Two-phase transfer learning: stable and needs far fewer images.
Class imbalance	Ignore it; collect more of the small classes	Class weights: smaller stages count more in the loss, at no extra cost.
Bloom inputs	Image only; readings only	Image and readings together: the image shows the stage and the readings show the conditions.
Bloom output	Linear output; predict a class of days	ReLU output of days: days cannot be negative and an open flower can be exactly zero.
Object detector	Train a YOLO model first	OWLv2 zero-shot first: works without hand-drawn boxes; YOLOv8 can be added later.
Backend	Separate server per component	One shared FastAPI backend for all four components: one app, one address.

4.4 Detailed Design

4.4.1 Growth stage classifier

The classifier is built on EfficientNetB0 pretrained on ImageNet, with its original 1,000-class output layer removed. A global average pooling layer reduces the base output to 1,280 values, which then pass through two dense layers of 512 and 256 units. To limit overfitting, each dense layer uses rectified linear unit (ReLU) activation, L2 weight regularisation, batch normalisation [10] and 30 per cent dropout [9]. The final layer has six softmax units, giving a probability for each stage. In total the model has 4,841,385 parameters, and its layers are listed in Table 4.4.

Table 4.4: Layers of the growth stage classifier

Layer	Output size	Notes
Input	224 × 224 × 3	Colour photograph
EfficientNet preprocessing	224 × 224 × 3	Scales pixel values
EfficientNetB0 base	7 × 7 × 1,280	ImageNet weights
Global average pooling	1,280	
Dense + batch normalisation + dropout	512	ReLU, L2, dropout 0.3
Dense + batch normalisation + dropout	256	ReLU, L2, dropout 0.3
Dense (output)	6	Softmax over the six stages

4.4.2 Bloom prediction model

The bloom model has two branches. The image branch consists of EfficientNetB0, global average pooling, a 128-unit dense layer with batch normalization and 50 per cent dropout, and a 32-unit layer. The sensor branch takes the three scaled readings, adds a little random noise during training only, and passes them through dense layers of 32 and 16 units. The two branches are then merged and fed through a 32-unit layer with 50 per cent dropout to a single output. This output uses ReLU activation so it can never be negative, which also means a plant already in bloom can be predicted as exactly zero days. The model has 4,220,436 parameters and is summarized in Table 4.5.

Table 4.5: Structure of the bloom prediction model

Branch	Layers	Output
Image	EfficientNetB0, global average pooling, Dense 128 + batch normalization + dropout 0.5, Dense 32	32 numbers describing the photo
Sensor	Gaussian noise 0.15, Dense 32 + dropout 0.3, Dense 16	16 numbers describing the readings
Head	Join both branches, Dense 32 + dropout 0.5, Dense 1 (ReLU)	Days until bloom

When a prediction is made, the number of days is rounded to a whole number and added to the date of the photograph to give the expected bloom date.

4.4.3 Multi-object pipeline

A photograph of a bench often shows several plants, or a single plant with both open flowers and new buds, and assigning one stage to the whole image would blur these together. The multi-object pipeline handles this in three steps. First, OWLv2 searches for four kinds of object using the text prompts “orchid plant”, “orchid flower bunch”, “orchid flower bud” and “orchid seed pod”, keeping any detection that scores above 0.15. Next, each box is enlarged by eight per cent on every side to preserve some context, and that region is cropped from the photograph. Finally, each crop is passed to the growth stage classifier. The response lists every object with its type, detection score, box and stage. The design also leaves room for a custom YOLOv8 detector to take over for plants, flower bunches and seed pods once enough boxed photographs are available.

4.4.4 API design

The component follows the REST style: each endpoint has a clear path, uses POST to send a photograph and GET to retrieve information, and returns JSON. Table 4.6 lists the endpoints.

Table 4.6: Endpoints of the component

Method and path	Input	Output
POST /api/v1/growth/identify	Image file	Stage, label, confidence, top three stages, description, care advice
POST /api/v1/growth/identify-objects	Image file	Number of objects; for each: type, score, box, stage
GET /api/v1/growth/stages	–	All six stages with description and care advice
GET /api/v1/growth/stage/{key}	Stage key	Description and care advice for one stage
GET /api/v1/growth/health	–	Whether the model is loaded
POST /api/v1/bloom/predict	Image, temperature, humidity, light intensity, optional date	Days until bloom, bloom date, input readings
POST /api/v1/bloom/predict-objects	As above	Days until bloom for each detected object
GET /api/v1/bloom/health	–	Whether the model is loaded

4.4.5 Request flow

Figure 4.2 shows what happens when a grower requests the growth stage of a plant. The app sends the photograph to the backend, which checks the file type, prepares the image and runs the model. The model returns six probabilities, and the backend selects the highest, attaches the care advice for that stage and sends the response back to the app. The app then displays the stage card and care tips.

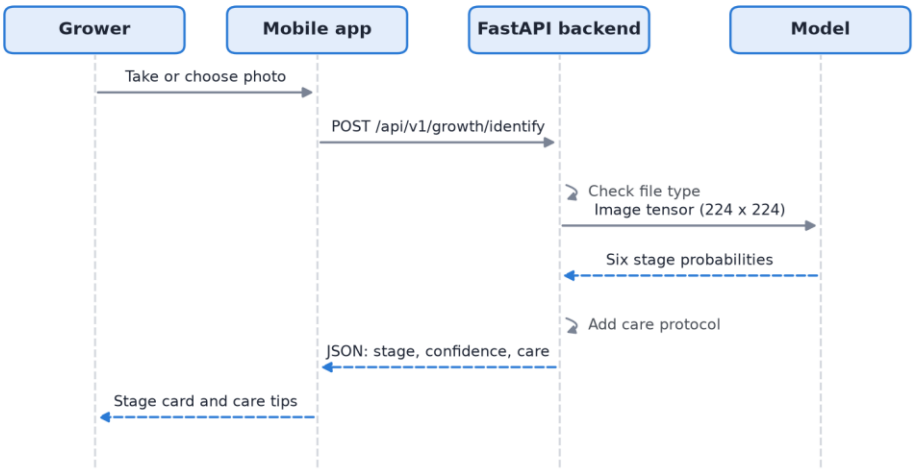


Figure 4.2: Sequence of a growth stage request

4.4.6 User interface design

The Growth Stage screen was designed to be easy for growers to use. A single large area displays photographs, with buttons for taking a picture or choosing one from the gallery. Once the analysis is done, a result card shows the stage name, a confidence bar and the top three stages, and below it a care card gives advice on water, light, temperature and fertilizer, each with a small icon. The bloom section contains three number fields and a Predict Bloom Date button, which is only enabled when a photograph and all three readings have been provided. A Share Results button sends the main results as text. Screenshots of the working screen are given in Appendix A.5.

4.4.7 Care protocol

The care advice is taken from a fixed table prepared together with growers and drawn from common Vanda care practice. It is shown in Table 4.7.

Table 4.7: Care advice returned for each stage

Stage	Water	Light	Temperature	Fertilizer
Vegetative	Water when nearly dry	Moderate-bright	24-30 °C	Balanced feed
Budding	Slightly increase watering	Bright filtered	24-29 °C	Bloom booster
Pre-Bloom	Consistent moisture	Steady bright	23-28 °C	Bloom formula
Full Bloom	Regular, avoid petals	Bright indirect	22-27 °C	Light support
Wilting	Reduce gradually	Moderate	20-26 °C	Switch to balanced
Seed Formation	Stable routine	Bright indirect	22-28 °C	Half strength

4.5 Standards and Professional Practice

The design follows established professional standards. The non-functional requirements in Table 4.2 are organized around the main quality characteristics of the ISO/IEC 25010 software quality model [15]: functional suitability, performance efficiency, usability, reliability, security and maintainability. The API follows REST conventions and returns standard Hypertext Transfer Protocol (HTTP) status codes, namely 200 for success, 400 for an unsupported file type, 422 for missing fields and 500 for server errors. The Python code follows the PEP 8 style guide, all work was tracked in Git with clear commit messages, and references follow the IEEE style.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The component was built using widely used open-source tools, which are listed in Table 5.1.

Table 5.1: Technology stack of the component

Layer	Technology	Purpose
Programming language	Python 3	Models, data preparation and backend
Deep learning	TensorFlow and Keras 3 [13]	Building, training and running the models
Data and evaluation	pandas, NumPy, scikit-learn [14], Pillow	Reading spreadsheets, splitting, metrics, image handling
Object detection	OWLv2 [7] through Hugging Face Transformers	Finding plants, flower bunches, buds and seed pods
Training platform	Google Colab with GPU	Training both models
Backend	FastAPI with Uvicorn	REST API shared by all four components
Mobile app	React Native 0.81 with Expo 54	Growth Stage screen on Android and iOS
Version control	Git and GitHub	Code history and team work
Editor	Visual Studio Code	Writing and debugging code

5.2 Component Implementation

The code is divided into small modules, each with a single responsibility. Table 5.2 lists the main files.

Table 5.2: Main source files of the component

File	Job
growth_stage/src/config.py	Stage names, descriptions, care advice and model settings
growth_stage/src/clean_dataset.py	Converts unreadable image formats and moves duplicate files to quarantine
growth_stage/src/preprocess.py	Lists images, splits them and feeds batches to the model
growth_stage/src/model.py	Builds the classifier and its training callbacks
growth_stage/src/train.py	Two-phase training, evaluation and saving of reports
growth_stage/src/predict.py	Loads the saved model and predicts the stage of a photograph
growth_stage/src/detect_and_predict.py	Multi-object pipeline: detect, crop and classify
bloom_prediction/src/*.py	The same structure for the bloom prediction model
shared/object_detection/	OWLv2 and YOLOv8 detector shared by both models
backend/app/api/routes/growth_stage.py	Growth stage API endpoints
backend/app/api/routes/bloom_prediction.py	Bloom prediction API endpoints
mobile/src/screens/GrowthStageScreen.js	Growth Stage screen of the mobile app

5.2.1 Data preparation code

The data loader reads every image in the six stage folders and records its stage. It then splits each stage into training, validation and test sets and saves the three lists as comma-separated values (CSV) files. A Keras data generator reads the images in batches, resizes them, applies the random training changes and converts each label into a one-hot vector. For the bloom model, the loader reads each dated spreadsheet, matches every row to its image file, drops rows with missing values or missing images, and scales the readings.

5.2.2 Training code

The training script builds the model, calculates class weights from the training set and trains in the two phases described in Section 3.3.4. Training is controlled by three callbacks: early stopping, which halts training when the validation loss stops improving and restores the best weights; a learning rate reducer, which divides the learning rate by five whenever progress stalls; and a checkpoint that saves the best model. Once training finishes, the script evaluates the model on the test set and saves the classification report, confusion matrix, training curves and model settings.

5.2.3 Prediction code

The predictor class loads the saved model once and keeps it in memory. Given a new photograph, it resizes the image, runs the model and returns the most probable stage together with its confidence, the top three stages, the stage description and the care advice. The bloom predictor additionally scales the readings with the saved scaler, rounds the predicted number of days and adds it to the photograph date to produce the bloom date.

5.2.4 Backend routes

Each route file loads its model only when the first request arrives, which keeps server start-up fast. Every upload is checked against a list of permitted image types, and any other file receives an HTTP 400 response. Valid images are written to a temporary file, passed to the predictor and then deleted. One complication is that the growth stage and bloom modules both contain a Python package named src. To prevent one from masking the other, each route file removes that package from Python's module cache once it has imported what it needs.

5.2.5 Mobile screen

The Growth Stage screen uses the Expo image picker to access the camera and gallery. When the grower taps Analyze, the screen sends the photograph to the identify endpoint and displays the result and care cards. Before sending a bloom prediction request, it checks that all three readings are valid numbers. Errors from the backend, such as a dropped connection, are shown as a message in place of a result, and the Share button opens the phone's share sheet.

5.3 Integration with the Other Components

Component 4 connects to the rest of the system in three ways.

- **Shared backend.** The growth stage and bloom routes are registered in the same FastAPI application as those of the other components, under the paths /api/v1/growth and /api/v1/bloom. The app reaches every component through a single server address defined in one configuration file.
- **Watering and fertilization (C3).** The growth stage tells C3 which fertilizer to apply. The care advice moves from a balanced feed in the vegetative stage, to a

bloom booster at budding and a bloom formula just before flowering, then to light support in full bloom, and back to a balanced feed after wilting.

- **Hybrid pollination (C1).** The bloom date tells C1 when a plant will have open flowers, which is when pollination becomes possible.

The components communicate through the JSON response of each endpoint. Because this response format was fixed in advance, each component could be developed and tested independently before being joined with the others.

5.4 Verification and Software Testing

5.4.1 Testing strategy

Testing was carried out at four levels, which are shown in Table 5.3.

Table 5.3: Testing levels

Level	What was tested	How
Model evaluation	Accuracy, precision, recall and F1 per stage; MAE, RMSE and R^2 for bloom; comparison with baselines	Test sets, TensorFlow and scikit-learn
Functional testing	Every endpoint with correct and wrong inputs	Python script sending real requests to a running backend
Performance testing	Response time of each type of prediction	Timed requests on a laptop CPU
User interface testing	Photo capture, input checks, result display, sharing	Manual tests with Expo Go

5.4.2 Test environment

The functional and performance tests were run against the backend on a Windows 11 laptop, using TensorFlow 2.20.0 with Keras 3.15.1 on the CPU only. Every test photograph came from the test sets, so none had been used during training.

5.4.3 Functional and performance test cases

Table 5.4 lists the 16 API test cases and their outcomes. Of these, 15 passed. The single failure concerns the response time of the multi-object endpoint, which is discussed in Sections 6.2 and 7.5.

Table 5.4: API test cases and results

ID	Test case	Expected result	Actual result	Status
T01	Growth stage health check	HTTP 200, model loaded	HTTP 200, model_loaded=True	Pass
T02	List growth stages	HTTP 200, six stages	HTTP 200, 6 stages	Pass
T03	Stage information by key	HTTP 200 with care protocol	HTTP 200	Pass
T04	Unknown stage key	Request rejected	HTTP 400	Pass
T05	Identify stage: response content	Stage, confidence, top three stages, care protocol	All fields present	Pass
T06	Identify stage on 60 test images (10 per stage)	Correct stage for most images	56 of 60 correct	Pass
T07	Identify stage: response time	Under 5 seconds	Median 422 ms, 95th percentile 638 ms	Pass
T08	Non-image upload to identify	HTTP 400	HTTP 400	Pass
T09	Multi-object identification	HTTP 200, list of detected objects with stages	HTTP 200, each detected object returned with its stage	Pass
T16	Multi-object identification: response time	Under 5 seconds	53 to 80 s per photograph (five photographs, model already loaded)	Fail
T10	Bloom health check	HTTP 200, model loaded	HTTP 200, model_loaded=True	Pass
T11	Bloom prediction: response content	Days until bloom and predicted bloom date	Both fields present	Pass
T12	Bloom prediction on 30 test images	Mean error about one day	30 answered, mean error 1.50 days	Pass
T13	Bloom prediction: response time	Under 5 seconds	Median 620 ms, 95th percentile 986 ms	Pass
T14	Bloom prediction with missing readings	HTTP 422	HTTP 422	Pass
T15	Non-image upload to bloom prediction	HTTP 400	HTTP 400	Pass

5.4.4 User interface test cases

The mobile screen was tested on 11 October 2026 against the cases in Table 5.5. The Expo web build of the app was run in Google Chrome using mobile device emulation (Pixel 7 profile, 412 × 915 pixels) and connected to the FastAPI backend, and each case was carried out with a Playwright script using the budding test photograph shown in Appendix A.5. In the browser, the camera option opens the device capture dialog, and the share case was checked by recording the content passed to the Web Share API. The backend-not-reachable case was simulated by blocking all requests

to the backend. The share case failed on the first run because the shared text left out the bloom date and two of the care values; the share function was corrected and the case passed on retest. All eight cases passed.

Table 5.5: User interface test cases

Case	Expected behavior	Actual result	Status
Take a photograph with the camera	The photograph appears and can be analyzed	Capture dialog opened; photograph shown and analyzed as Budding Stage	Pass
Choose a photograph from the gallery	The photograph appears and can be analyzed	Photograph shown in the preview and sent for analysis	Pass
Analyze a photograph	Stage, confidence, top three stages and care card are shown	Budding Stage at 99.9% confidence, top three stages and care card shown	Pass
Predict bloom with a missing reading	The Predict Bloom Date button stays disabled	Button disabled; no request sent	Pass
Predict bloom with a reading that is not a number	A message asks for valid numbers and nothing is sent	Message "Enter valid numbers for temperature, humidity, and light." shown; no request sent	Pass
Predict bloom with valid inputs	Days until bloom and the bloom date are shown	6 days until bloom and bloom date 2026-10-17 shown	Pass
Share results	The phone share sheet opens with the stage, care advice and bloom date	Share sheet opened with stage, confidence, care advice and bloom date	Pass
Backend not reachable	An error message is shown instead of a result	Error message "Failed to fetch" shown instead of a result	Pass

5.5 Deployment and Operation

The backend runs as a single FastAPI service on a virtual private server, which the mobile app reaches through a domain name. Because the server address is set in one configuration file in the app, it can be changed without touching any of the screens. The trained models are stored as .h5 files alongside their code and are loaded when the first request arrives.

Since the models were saved with Keras 3, the server must run TensorFlow 2.16 or later. Each model exposes a health endpoint that reports whether it has been loaded, which makes it simple to check the server after an update. Updating a model only requires placing the new .h5 file in the models folder and restarting the server; neither the API nor the app needs to change.

6 RESULTS AND VALIDATION

6.1 Research Results

6.1.1 Growth stage classification

On the 1,768 test images, the growth stage classifier achieved an **accuracy of 0.940** and a **macro F1 score of 0.94**. The most common stage in the test set is full bloom, with 346 images, so a model that always answered "full bloom" would score just 0.196. The classifier therefore improves on the baseline by 0.744, making it almost five times as accurate. Table 6.1 gives the results for each stage.

Table 6.1: Growth stage classifier: results for each stage on the test set

Stage	Precision	Recall	F1 score	Test images
Vegetative	0.94	0.83	0.88	280
Budding	0.98	0.97	0.98	300
Pre-Bloom	0.94	0.94	0.94	311
Full Bloom	0.95	0.94	0.95	346
Wilting	0.94	0.93	0.93	220
Seed Formation	0.89	1.00	0.94	311
Macro average	0.94	0.94	0.94	1,768
Accuracy			0.94	1,768

Five of the six stages reached a recall of 0.93 or higher. Seed formation was identified in every case (recall 1.00) and budding in almost every case (recall 0.97). Figure 6.1 shows the recall for each stage alongside the baseline.

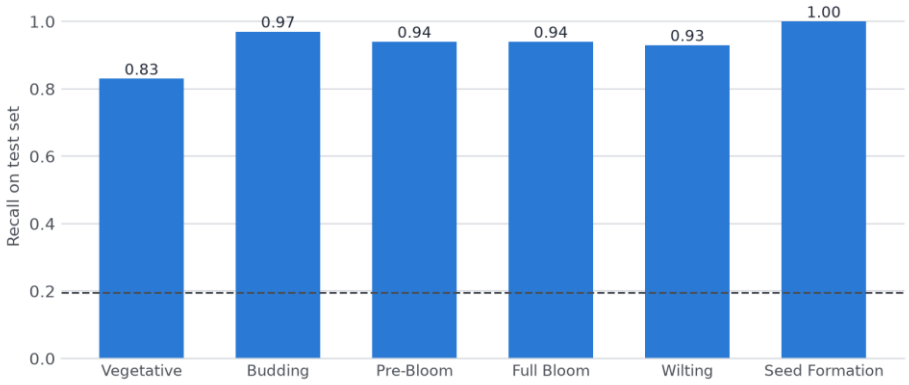


Figure 6.1: Recall of each stage on the test set; the dashed line is the majority-class baseline of 0.196

The confusion matrix in Figure 6.2 reveals where the mistakes occur. Vegetative is the weakest stage, with a recall of 0.83: of its 280 test images, 32 were predicted as

seed formation. This is the only confusion above ten per cent, and it is understandable. Neither stage has open flowers, and in both the photograph is dominated by leaves, stem and roots, so a small seed pod can easily resemble new growth. The remaining mistakes occur between consecutive stages, such as pre-bloom and full bloom, or full bloom and wilting.

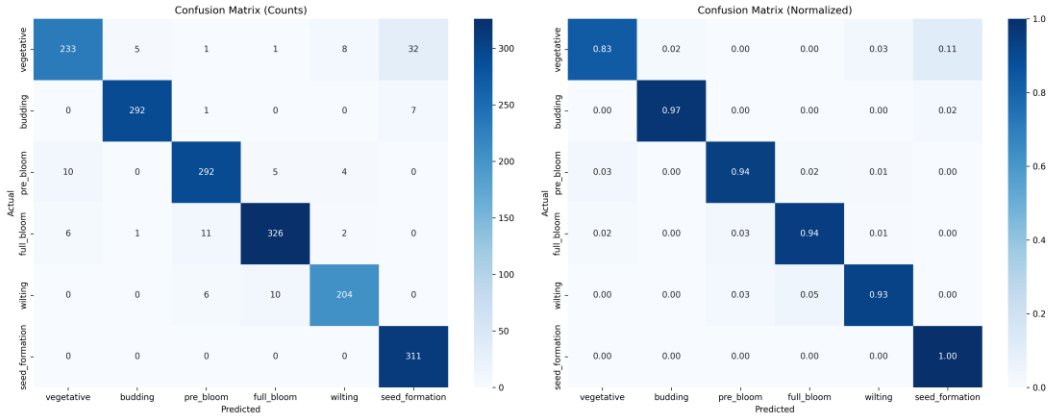


Figure 6.2: Confusion matrix of the growth stage classifier on the test set, as counts and as proportions of each row

Figure 6.3 shows how the model learned. In phase one, with the base frozen, validation accuracy climbed to around 0.90. Phase two began at epoch 50 and lifted validation accuracy further, to about 0.95. The gap between training and validation accuracy remained small throughout, indicating that dropout, regularisation and random image changes kept overfitting in check.

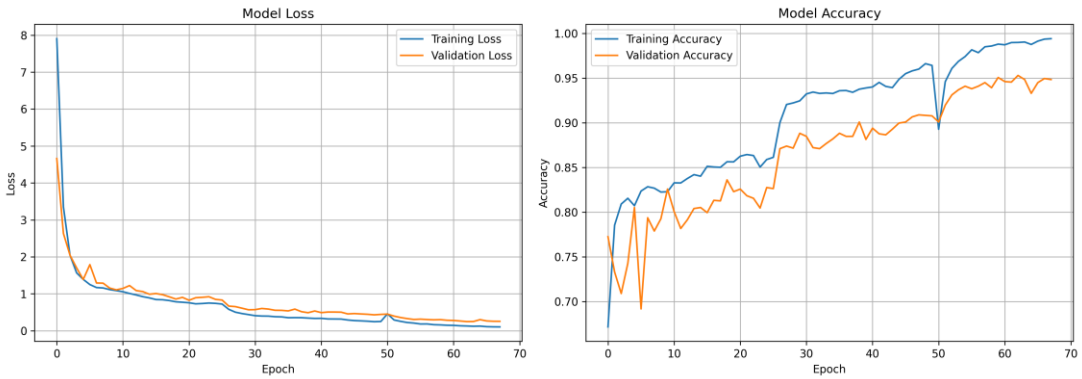


Figure 6.3: Training and validation loss and accuracy of the growth stage classifier; phase two starts at epoch 50

6.1.2 Bloom prediction

On the 1,839 test images, the bloom model achieved a **mean absolute error of 0.92 days**, a **root mean squared error of 1.41 days** and a **coefficient of determination**

of 0.938. Always predicting the training-set average would give a mean absolute error of 4.86 days, so the model cuts the error roughly fivefold. Table 6.2 summarizes the results.

Table 6.2: Bloom prediction results on the test set

Measure	Bloom model	Always predict the average
Mean absolute error	0.92 days	4.86 days
Root mean squared error	1.41 days	–
Coefficient of determination (R^2)	0.938	0
Test images	1,839	1,839

Figure 6.4 plots predicted against actual days. Most points sit close to the diagonal, meaning the predictions are close to the true values. The largest errors appear at the long end, where plants 20 to 23 days from bloom are predicted a few days too early. This happens because the bloom record contains few such images (Figure 3.3), so the model leans towards the more common shorter waits. Plants already in bloom are predicted at or near zero.

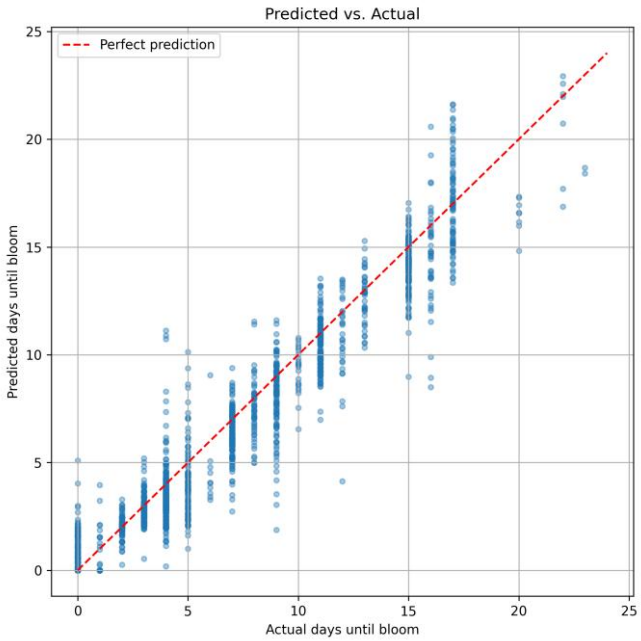


Figure 6.4: Predicted against actual days until bloom on the test set; the dashed line is a perfect prediction

Figure 6.5 shows the training curves. The validation loss stays below the training loss throughout, which is expected for this model: dropout, noise and random image

changes make training harder, but they are switched off when the validation set is evaluated.

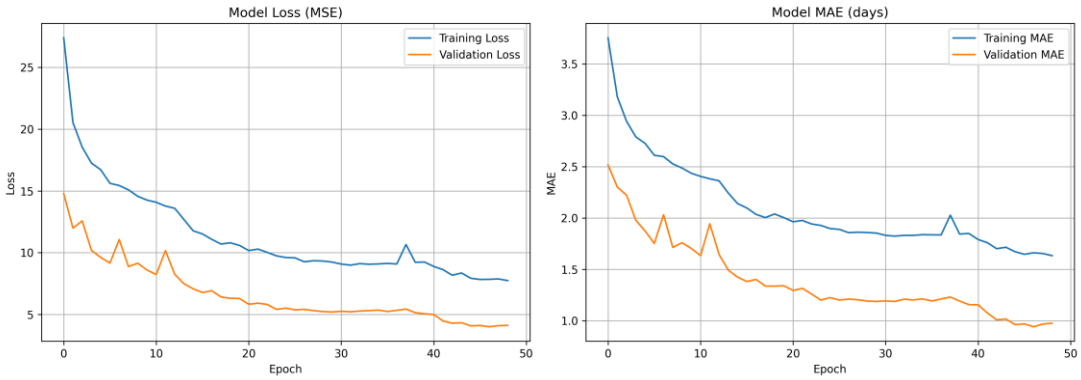


Figure 6.5: Training and validation loss and mean absolute error of the bloom prediction model

6.2 Product and Technical Results

The API was tested with the backend running on a laptop CPU, after the first request had loaded the models. Across 60 requests, a single-plant growth stage prediction took a median of 422 milliseconds (95th percentile 638 ms). Across 30 requests, a bloom prediction took a median of 620 ms (95th percentile 986 ms). Both are comfortably within the five-second limit set in NFR3.

In the same run, the classifier identified the correct stage for 56 of the 60 test photographs, and the bloom model's average error across 30 test photographs was 1.50 days. This is slightly higher than the full test-set figure of 0.92 days, partly because the sample is small and partly because the API rounds each prediction to whole days.

The multi-object endpoint worked correctly and returned every detected object with its stage, but it was slow, taking between 53 and 80 seconds per photograph on the laptop CPU. OWLv2 is a large vision transformer and needs a GPU to run quickly. For this reason, the Growth Stage screen in the app uses the single-plant endpoint, while the multi-object endpoint remains available through the API for occasional use. Sample API responses are given in Appendix A.

6.3 User and Stakeholder Validation

The care advice in Table 4.7 was prepared together with growers, and the six stages were chosen to match the way growers themselves describe their plants. To validate the finished component, one Vanda grower used the app and gave feedback on its usefulness, its user interface and the integrated system.

Overall, the grower was satisfied with the component. They found the bloom prediction very helpful, because knowing when plants will bloom allows them to estimate how many flowers they can sell on a specific day. They found the user interface clear and easy to use. They also said that keeping all four components of the system in one app is very useful, because it means they do not need separate tools for each task. No changes to the component were requested as a result of this feedback.

6.4 Integration Results

The growth stage and bloom routes run alongside the other components in the same FastAPI application without conflict. The module isolation described in Section 5.2.4 allows both models to be loaded on the same server, and the health endpoints of both models confirmed that they were loaded (test cases T01 and T10). Because the stage output follows the agreed JSON format, the watering component can read the stage, and the bloom date is available to the pollination component.

6.5 Summary of Findings

Table 6.3: Summary of the main results

Measure	Result	Baseline or target	Met?
Growth stage accuracy	0.940	0.196 (majority class)	Yes
Growth stage macro F1	0.94	–	Yes
Bloom MAE	0.92 days	4.86 days (average)	Yes
Bloom R^2	0.938	0	Yes
Single-plant response time	422 ms median	Under 5 seconds	Yes
Bloom response time	620 ms median	Under 5 seconds	Yes
Multi-object response time	53–80 s	Under 5 seconds	No
API test cases passed	15 of 16	All	Partly

7 DISCUSSION

7.1 Interpretation of Findings

A phone photograph is enough to read the growth stage. Using nothing more than an ordinary photograph, the classifier reached 0.940 accuracy across six stages. This answers RQ1: transfer learning can distinguish the Vanda stages and performs far better than guessing. The stages it does confuse are the same ones a person might confuse, which suggests the model has picked up the same visual cues that growers rely on.

Two-phase training gave the last improvement. Training only the new layers brought validation accuracy to about 0.90, and unfreezing the last layers of the base added roughly five more percentage points. Adapting the higher-level features to orchid photographs was clearly worthwhile, and doing so only after the new layers had settled kept training stable.

Clean data matters as much as the model. Photographs skipped because of their file type, and identical photographs carrying two different labels, do not show up in a training log, yet they limit what any model can learn. Fixing them with a script was an important step before the final training run.

A photograph plus three readings predicts bloom to within about a day. The bloom model's average error was 0.92 days, against 4.86 days for the average-value baseline. This answers RQ2: an error of around one day is small enough for a grower to plan a sale or a shipment.

Results reach the grower quickly. Single-plant stage and bloom predictions both return in well under a second on a laptop, which answers RQ3 for the main path through the app. The multi-object path, however, is too slow on a CPU and needs further work.

7.2 Comparison with Literature, Baselines and Existing Solutions

Table 7.1: Comparison with existing work

Aspect	Optical sensing [11]	OrchiZen [12]	This component
Main input	Hyperspectral and fluorescence sensors	Camera and sensors	Phone photograph and three readings
Stages	Growth parameters	Five age groups	Six flowering-cycle stages
Bloom prediction	Yes, 21 days ahead	No	Yes, average error 0.92 days
Care advice	No	Fertilizer amounts	Water, light, temperature, feed
Cost to grower	High	Medium	Low

As Table 7.1 shows, this component trades the detailed plant measurements of the optical approach [11] for low cost: it needs no special instrument, only a phone. Compared with OrchiZen [12], it tells the grower where a plant is in its flowering cycle rather than how old it is, which is the information that both feeding and selling decisions depend on, and it adds the bloom prediction that OrchiZen lacks. Measured against its own baselines, the classifier is almost five times better than guessing the most common stage, and the bloom model reduces the error of the average-value baseline roughly fivefold.

7.3 Research Contribution

- A labelled dataset of six Vanda growth stages taken in Sri Lankan shade houses.
- Evidence that an ImageNet-pretrained EfficientNetB0 with two-phase training can classify these six stages with 0.940 accuracy.
- A two-branch model that predicts days until bloom from a photograph and three readings with an average error of 0.92 days.
- A pipeline that uses an open-vocabulary detector to classify each plant and flower bunch in a crowded photograph without any hand-drawn boxes.

7.4 Product Contribution

- A working Growth Stage screen in the group's mobile app that gives the stage, confidence, top three stages and care advice from one photograph.
- A bloom date feature that turns three readings and a photograph into a calendar date.

- Eight REST endpoints in the shared backend that other components and future tools can use.
- Stage and bloom outputs that connect the watering and pollination components to the real state of each plant.

7.5 Limitations and Threats to Validity

Limitations.

- **Germination and seedling stages are missing.** There were not enough usable photographs of these stages to include them.
- **Flower life and seed collection time are not predicted.** Doing so would require records that follow the same flowers and pods over a longer period.
- **No check for non-orchid photographs.** The classifier always returns one of the six stages, even when the photograph has nothing to do with orchids.
- **Long waits are under-predicted.** Plants 20 or more days from bloom are predicted a few days early because such examples are scarce in the data.
- **Multi-object detection is slow on a CPU.** Without a GPU it takes about a minute per photograph.
- **Readings are entered by hand.** A typing mistake in the temperature, humidity or light value will change the bloom prediction.

Threats to validity. All photographs were taken in a small number of Sri Lankan shade houses, using phone cameras in daylight, so the models may be less accurate in other locations, with other cameras or in poor light. The bloom record spans only a few weeks between July and August 2026, which means the bloom model has not yet been tested in other seasons. The user interface has also not yet been validated with a large group of growers.

7.6 Implications

For growers, these results mean that a dependable second opinion on growth stage, together with clear care advice, is available from a phone they already own. For new growers in particular, it puts within reach a skill that normally takes years to develop. Within the integrated system, the stage and bloom outputs allow the watering and

pollination components to respond to the actual state of each plant rather than to a fixed schedule. For research, the results show that a modest dataset and a small pretrained network are sufficient for a useful phenology tool, without the need for expensive sensors.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The component helps growers feed each plant correctly and know in advance when it will be ready for sale. Table 8.1 sets out the main users and what each gains.

Table 8.1: Target users and value

User	Need	Value from the component
Small Vanda growers	Correct feeding without years of experience	Stage and care advice from a phone photo, free of charge
Commercial nurseries	Plan which plants will flower for sale	Bloom dates for many plants, records over time
Exporters	Flowering stock ready on a shipping date	Advance notice of bloom dates
New growers and students	Learn to read Vanda stages	Stage names, descriptions and care for every photo
Agricultural advisers	Support many growers at once	Consistent stage readings that can be shared

8.2 Market and Competitive Position

Sri Lanka's floriculture export sector is growing, and orchids are among its higher-value products. Most Vanda growers run small or medium businesses that cannot afford scientific instruments. Table 8.2 compares the component with the alternatives available to them.

Table 8.2: Competitive position

Alternative	Cost	Stage	Bloom date	Care advice
Manual inspection	Labour and experience	Yes, not consistent	Rough guess	From experience
General plant ID apps	Free or low	No	No	General only
Research sensor systems	High	Indirect	Yes	No
This component	Free app; optional sensors	Yes, six stages	Yes	Yes, per stage

8.3 Business and Sustainability Model

The component is offered as part of the integrated orchid care app rather than as a standalone product. Table 8.3 outlines a possible business model; pricing has not yet been tested with growers.

Table 8.3: Possible business model

Tier	Features	Who pays
Free	Stage, care advice and bloom date from photographs and typed readings	No one; supports adoption by small growers
Nursery subscription	History of stage and bloom per plant, reports for many benches	Commercial nurseries
Hardware bundle	Sensor node that sends readings automatically (shared with the watering component)	Growers who want automation
Partnerships	Bloom forecasts for planning shipments; training for new growers	Exporters, extension services

Running costs are low. The models are small enough to run on a single modest server, and the stage and bloom features require no extra hardware, which keeps the free tier sustainable.

8.4 Scalability, Intellectual Property and Compliance

Scalability. Each prediction takes under a second on a CPU, so a single server can handle many growers, and more can be supported by adding servers behind a load balancer. Heavy use of the multi-object feature, however, would require a GPU server.

Intellectual property. The code and models were developed by the group at SLIIT. The pretrained EfficientNetB0 weights, TensorFlow and OWLv2 are used under their open-source licences, and the photographs belong to the research and were taken with the growers' permission.

Compliance. The component does not store personal data. Should user accounts be added in a commercial version, the system will need to comply with the Sri Lanka Personal Data Protection Act, No. 9 of 2022.

8.5 Contribution to the Sustainable Development Goals

Table 8.4: Contribution to the Sustainable Development Goals [16]

Goal	Target	How the component contributes
SDG 12: Responsible consumption and production	12.2 Efficient use of natural resources	Fertilizer is matched to the real stage of each plant instead of a fixed schedule, so less is wasted.
SDG 8: Decent work and economic growth	8.2 Productivity through technology	New growers can read plant stages without years of experience, and bloom dates help plan sales.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This research set out to help Vanda growers with two judgements they normally make by eye: what growth stage a plant has reached and when it will bloom. The component developed here recognizes six stages of the Vanda flowering cycle from an ordinary phone photograph with an accuracy of 0.940 and a macro F1 score of 0.94, against a baseline of 0.196. It also predicts the number of days until bloom from a photograph and three readings, with an average error of 0.92 days and an R^2 of 0.938.

Beyond this, the component locates and classifies each plant, flower bunch, bud and seed pod in a crowded photograph, provides care advice for every stage, and delivers its results through the group's mobile app. Within the integrated system, it tells the watering component which feed to use and tells the pollination component when flowers will be open. In doing so, the work fills the gap identified in the literature: a low-cost tool, built specifically for Vanda, that follows the flowering cycle and predicts bloom.

9.2 Achievement of Objectives

Table 9.1: Achievement of the specific objectives

Objective	Evidence	Status
SO1: Six-stage dataset	11,562 labelled images (Section 3.2.1)	Achieved
SO2: Stage classifier beats baseline	0.940 accuracy against 0.196 (Section 6.1.1)	Achieved
SO3: Bloom record and model	0.92 days MAE against 4.86 days (Section 6.1.2)	Achieved
SO4: Several objects in one photo	Multi-object pipeline works (T09); speed needs a GPU (T16)	Partly achieved
SO5: API and mobile app	Eight endpoints, Growth Stage screen, 15 of 16 tests passed (Section 5.4)	Achieved
SO6: Share outputs with other components	Stage used by C3, bloom date by C1 (Section 5.3)	Achieved

9.3 Individual Research and Product Contributions

My individual research contribution consists of a six-stage Vanda dataset, a stage classifier with 0.940 accuracy, a bloom prediction model with an average error of

0.92 days, and a method for classifying each object in a crowded photograph without hand-drawn boxes. My individual product contribution is the Growth Stage screen of the mobile app, eight API endpoints in the shared backend, and the stage and bloom outputs used by the watering and pollination components.

9.4 Future Directions

- **Add the germination and seedling stages** by collecting more photographs, so the component covers the whole life of the plant.
- **Predict flower life and seed collection time** by following the same flowers and seed pods over time.
- **Learn differences between Vanda hybrids** by recording the hybrid name with each photograph.
- **Collect more long-wait examples** so the bloom model is more accurate three weeks ahead.
- **Add a check for non-orchid photographs**, like the one used by the pollination component.
- **Read the sensor values automatically** from the shade-house sensor nodes used by the watering component.
- **Speed up multi-object detection** by training the planned YOLOv8 detector or by running the detector on a GPU.

REFERENCES

[1] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.

[2] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Miami, FL, USA, 2009, pp. 248–255.

[3] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Trans. Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[4] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?” in *Advances in Neural Information Processing Systems 27 (NeurIPS)*, 2014, pp. 3320–3328.

[5] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.

[6] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, Art. no. 1419, 2016.

[7] M. Minderer, A. Gritsenko, and N. Houlsby, “Scaling open-vocabulary object detection,” in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, New Orleans, LA, USA, 2023.

[8] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[10] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, Lille, France, 2015, pp. 448–456.

[11] A. Sungheetha, R. Rajesh Sharma, B. P. Sreeja, S. Mahapatra, R. Thakur, and R. C. Karpagalakshmi, “Multi-modal optical sensing fusion for quantitative monitoring of orchid parameters: Analysis of commercial cultivation in Asia,” in *Proc. 2025 IEEE Int. Conf. Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER)*, 2025, doi: 10.1109/DISCOVER66922.2025.11259021.

[12] D. Wijendra, V. Jayasinghearachchi, O. A. P. Dilshan, H. M. K. C. B. Herath, Y. M. T. N. S. Yapa, and K. D. M. M. Rathnasiri, "OrchiZen: Hybrid integrated smart farming system for orchid plantations," in *Proc. 2025 IEEE Int. Conf. Automatic Control and Intelligent Systems (I2CACIS)*, 2025, doi: 10.1109/I2CACIS65476.2025.11101702.

[13] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Savannah, GA, USA, 2016, pp. 265–283.

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[15] International Organization for Standardization, *ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*. Geneva, Switzerland: ISO, 2011.

[16] United Nations, *Transforming Our World: The 2030 Agenda for Sustainable Development*. New York, NY, USA: United Nations, 2015.

GLOSSARY

Baseline – A very simple method, such as always guessing the most common answer, used to judge whether a model is really useful.

Batch normalization – A layer that keeps the numbers inside a network in a stable range, which makes training faster and more stable.

Class weight – A number that makes mistakes on smaller classes count more during training.

Confusion matrix – A table that shows how often each true class is predicted as each class.

Dropout – A training method that switches off random parts of a network so that it does not memorize the training data.

Epoch – One full pass of the training data through the model.

Fine-tuning – Further training of some layers of a pretrained network on new data with a small learning rate.

Monopodial – A growth habit in which a plant grows from a single stem that keeps growing upwards, as in Vanda.

Overfitting – When a model learns the training data too closely and performs worse on new data.

Phenology – The study of the stages in the life cycle of a plant, such as budding and flowering.

Shade house – A structure covered with shade netting in which orchids are grown.

Transfer learning – Reusing a model trained on one large task as the starting point for a new task.

Zero-shot detection – Finding objects described by text without training on boxed examples of those objects.

APPENDICES

Appendix A Extended Results and Technical Details

A.1 Sample API answers

Figures A.1 and A.2 show sample answers from the two main endpoints of the component. They were captured on 11 October 2026 from the interactive API documentation (Swagger UI) of the FastAPI backend, using the budding test photograph shown in Appendix A.5. The bloom request used 30 °C, 74 per cent humidity and 4,207 lux.

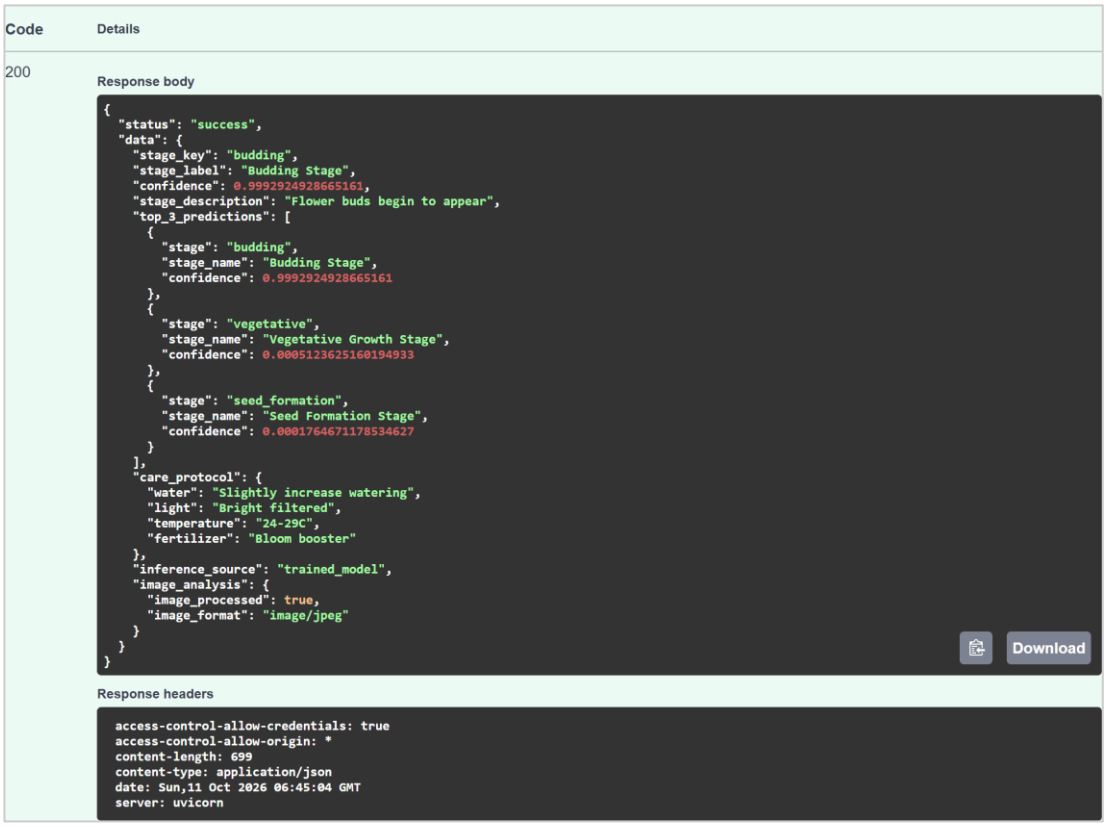


Figure A.1: Response of POST /api/v1/growth/identify for the budding test photograph (original in color)



Figure A.2: Response of POST /api/v1/bloom/predict for the same photograph at 30 °C, 74 per cent humidity and 4,207 lux (original in color)

A.2 Growth stage classifier code

```python
def create_cnn_model(input_shape: Tuple[int, int, int] = (224, 224, 3),
                     num_classes: int = 7,
                     model_type: str = 'efficientnet') -> tf.keras.Model:
    """
    Create a CNN model for growth stage classification.
    """
    # Create base model
    base_model = create_base_model(model_type, input_shape)
    base_model.trainable = False

    # Build the complete model
    inputs = tf.keras.Input(shape=input_shape)

    # Preprocess input based on model type
    if model_type == 'efficientnet':
        x = tf.keras.applications.efficientnet.preprocess_input(inputs)
    elif model_type == 'mobilenet':
        x = tf.keras.applications.mobilenet_v2.preprocess_input(inputs)
    elif model_type == 'resnet':
        x = tf.keras.applications.resnet50.preprocess_input(inputs)
    else:
        x = inputs / 255.0

    # Base model
    x = base_model(x, training=False)

    # Global pooling
    x = layers.GlobalAveragePooling2D()(x)

    # Dense layers with dropout
    x = layers.Dense(512, activation='relu', kernel_regularizer='l2')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.3)(x)

    x = layers.Dense(256, activation='relu', kernel_regularizer='l2')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.3)(x)

    # Output layer
    outputs = layers.Dense(num_classes, activation='softmax')(x)

    model = models.Model(inputs, outputs, name=f'vanda_growth_{model_type}')

    # Compile model with simple metrics
    model.compile(
        optimizer=Adam(learning_rate=MODEL_CONFIG['learning_rate']),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

A.3 Bloom prediction model code

```python
def create_bloom_model(input_shape: Tuple[int, int, int] = (224, 224, 3),
                       num_tabular_features: int = 3,
                       model_type: str = 'efficientnet') -> tf.keras.Model:
    """
    Create a two-branch model: image + tabular sensor readings -> days
    until bloom (regression).
    """
    base_model = create_base_model(model_type, input_shape)
    base_model.trainable = False

    # Image branch
    image_input = tf.keras.Input(shape=input_shape, name='image_input')

    if model_type == 'efficientnet':
        x = tf.keras.applications.efficientnet.preprocess_input(image_input)
    elif model_type == 'mobilenet':
        x = tf.keras.applications.mobilenet_v2.preprocess_input(image_input)
    elif model_type == 'resnet':
        x = tf.keras.applications.resnet50.preprocess_input(image_input)
    else:
        x = image_input / 255.0

    x = base_model(x, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(128, activation='relu', kernel_regularizer='l2')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.5)(x)
    image_embedding = layers.Dense(32, activation='relu')(x)


    tabular_input = tf.keras.Input(shape=(num_tabular_features,), name='tabular_input')
    t = layers.GaussianNoise(0.15)(tabular_input)
    t = layers.Dense(32, activation='relu', kernel_regularizer='l2')(t)
    t = layers.Dropout(0.3)(t)
    t = layers.Dense(16, activation='relu')(t)

    # Combine and regress. ReLU output because the target (days until bloom)
    # is never negative - this lets the network represent "already bloomed"
    # as an exact 0 instead of relying on predict.py's post-hoc clipping.
    combined = layers.Concatenate()([image_embedding, t])
    combined = layers.Dense(32, activation='relu', kernel_regularizer='l2')(combined)
    combined = layers.Dropout(0.5)(combined)
    output = layers.Dense(1, activation='relu', name='days_until_bloom')(combined)

    model = models.Model(
        inputs=[image_input, tabular_input],
        outputs=output,
        name=f'vanda_bloom_{model_type}'
    )

    model.compile(
        optimizer=Adam(learning_rate=MODEL_CONFIG['learning_rate']),
        loss='mse',
        metrics=['mae']
    )

    return model
```

A.4 Source code repository

The full source code of the integrated system, including this component, is available at https://github.com/dinith114/R26-SE-018.git.

A.5 Application screenshots

Figures A.3 to A.6 show the Growth Stage screen running in the Chrome browser in mobile view (Pixel 7 emulation), connected to the FastAPI backend. A test photograph of a budding Vanda was used. For the bloom prediction, the inputs were set to 30 °C, 74 per cent humidity and 4,207 lux, which are typical values in the bloom record.

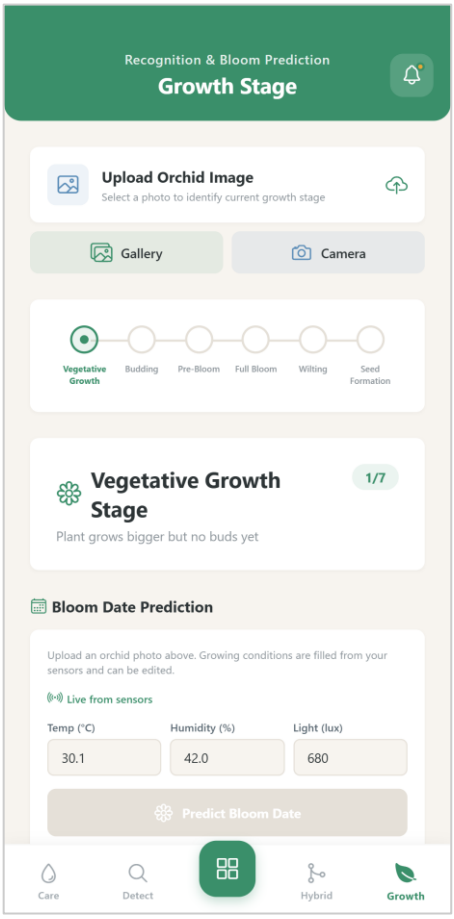


Figure A.3: Growth Stage screen before a photograph is chosen, showing the Gallery and Camera options and the live sensor readings

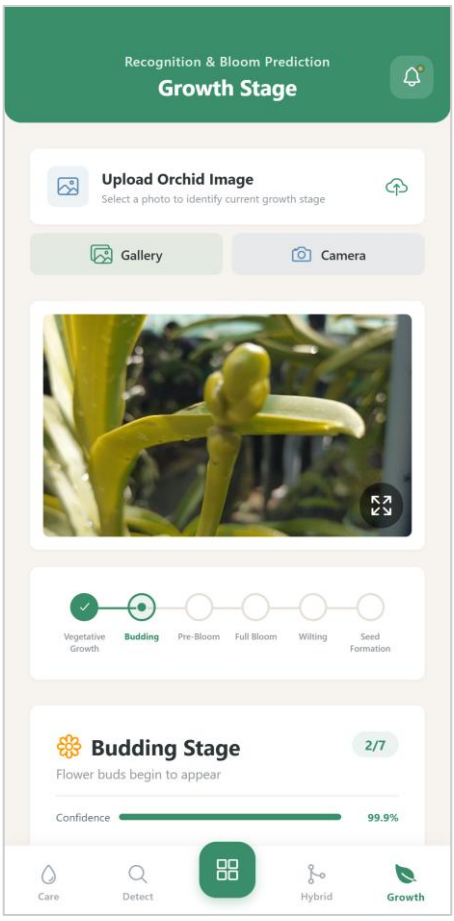


Figure A.4: Selected photograph of a budding Vanda and the predicted stage, Budding Stage, at 99.9 per cent confidence

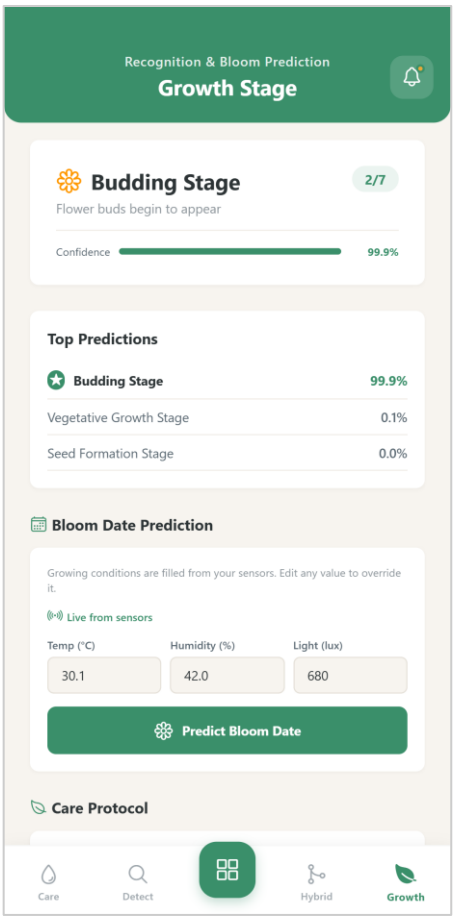


Figure A.5: Stage result with the confidence bar, the top three predictions and the bloom date inputs

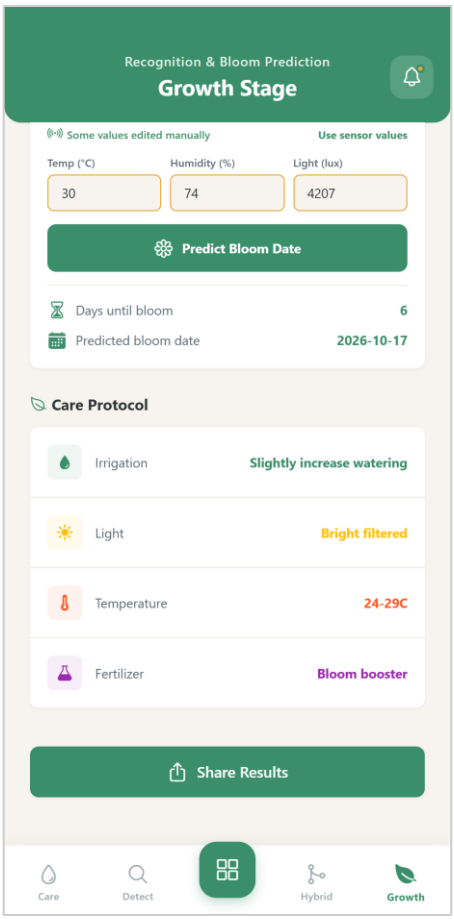


Figure A.6: Bloom date prediction of six days, the care protocol for the budding stage and the Share Results button

Appendix B Academic Integrity and AI Use Disclosure

Generative AI was used in this work as described in Table B.1. I checked every result, number, citation and piece of code against the project files and remained responsible for the content of this thesis.

Table B.1: Use of AI tools

Tool	Purpose	How it was checked	Parts affected
Claude (Anthropic), through Claude Code	Drafting and editing text in simple English; drawing charts and diagrams; formatting the document	Every number compared with the model reports and test outputs; text reviewed and edited by me	All chapters; Figures 1.1, 2.1, 3.1–3.3, 4.1, 4.2, 6.1; Table 5.4